Energy Efficiency in Railway Scheduling

Paul Tennigkeit, Georgi Abshilava

July 2026

1 Introduction

2 Encoding Energy Efficiency in ASP

2.1 Baseline Encoding

Flatland is a framework for simulating railway networks. A Flatland environment consists of a two dimensional plane made up of different types of cells, some of which containing different types of interconnected tracks, and agents moving around on them from a starting point to an end point. Agents have an earliest departure and arrival time, and a pre-defined speed, represented by an inverse value. To limit the time of the simulation, a global time frame is set.

The baseline ASP encoding handles all essential train and environment behavior, including defining legal transitions and movements, possible directions, and actions.

Directions There are four possible directions agents can go and point towards: North, south, east and west. Each is defined as a fact:

```
dir(n). dir(s). dir(e). dir(w).
```

Movements Each possible movement is defined as a fact:

```
moving(move_forward).
moving(move_left).
moving(move_right).
```

`delta()` defines what it means to "move" in each direction:

```
% The origin is located in the top left in Flatland, and
% the coordinates increment downwards

delta(A,s,1,0)  :- moving(A).
delta(A,n,-1,0) :- moving(A).
delta(A,e,0,1)  :- moving(A).
delta(A,w,0,-1) :- moving(A).
```

Transitions In order to represent different types of tracks, `transitions()` predicates are defined for each:

```
% ID  : track ID
% Dir1: pointing direction of agent
% Act : associated action with movement towards exit
% Dir2: exit direction
transition(ID, Dir1, Act, Dir2).
```

Due to the way this predicate is built, each track type is defined multiple times depending on its number of exit directions (e.g. straight tracks have two possible exits, north and south, and are, therefore, defined twice for each possible exit).

Actions The `action()` predicate is used as an interface between the Flatland visualization engine and the Clingo logic:

```
% train(ID): A train with an ID
% A        : An action
% T        : The time step action A is taken
action(train(ID),A,T) :- chosen(ID,T,A,_,_,_,_,_).
action(train(ID),move_forward,Dep) :- start(ID,_,Dep,_), Dep > 0.
action(train(ID),wait,T) :- start(ID,_,Dep,_), Dep > 0, T = 0..Dep-1.
```

Collision Constraints Like in real life, collisions between trains is to be avoided. To achieve this, the following constraints are employed:

```
% Trains can not occupy the same cell at the same time
:- stay(ID1,X,Y,T), stay(ID2,X,Y,T), ID1 < ID2.

% Trains can not swap their position
:- stay(ID1,X1,Y1,T), stay(ID2,X2,Y2,T),
   stay(ID1,X2,Y2,T+1), stay(ID2,X1,Y1,T+1), ID1 < ID2.
```